



BITS Pilani

Pilani Campus

Large Language Models for Gen AI

Dr. Monali Mavani

Large Language Models for Gen AI



Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Agenda



- Prompt Components & Design
- Prompting Paradigms: Zero-shot, Few-shot, In-Context Learning
- Structured Reasoning : Chain-of-Thought (CoT), ToT, Self consistency
- Iterative Reasoning : ReACT, Self-Reflection / Critique
- Prompt Engineering as System Design : Prompt Chaining & Control Flow, Query Decomposition, Routing / Query Classification, Function-Calling
- Structured Data Handling
 - Input — Handling Tables with LLMs (serialize / image / query)
 - Output — Verification, Structured Generation, Constrained Decoding
- Automated Prompt Optimization
- Test-Time Compute Scaling
- Context Engineering

Instruction prompting



Instruction tuning is what *made* instruction prompting work. Base models don't reliably follow instructions. We fine-tuned them specifically to follow natural-language instructions. *Because* of that tuning, you can now just *tell* the model what to do in plain language — that act of telling is "instruction prompting." The technique exists only because the models were trained to make it work.

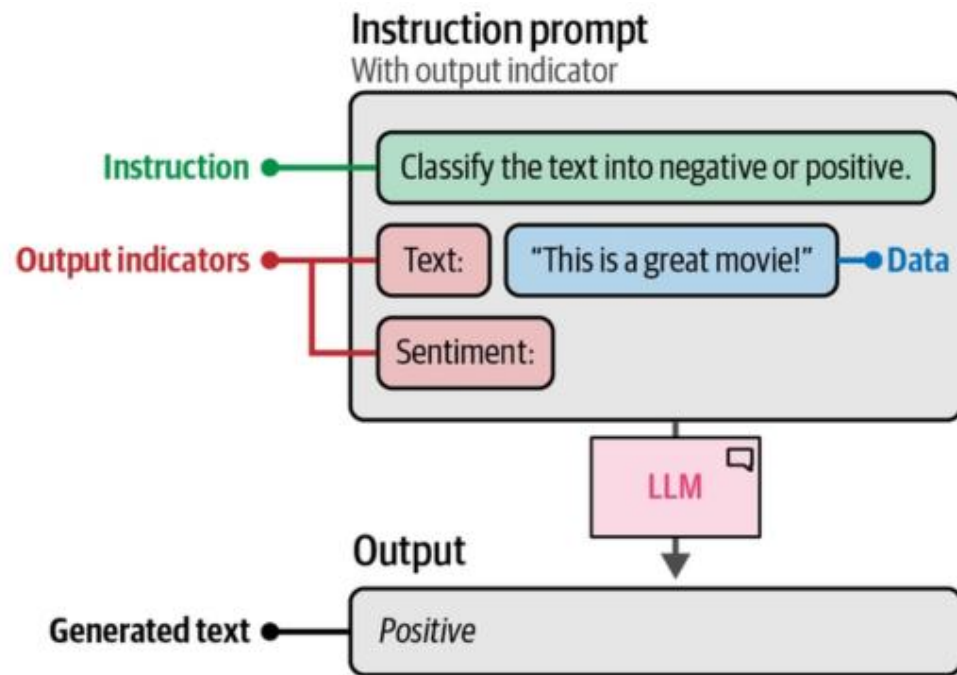
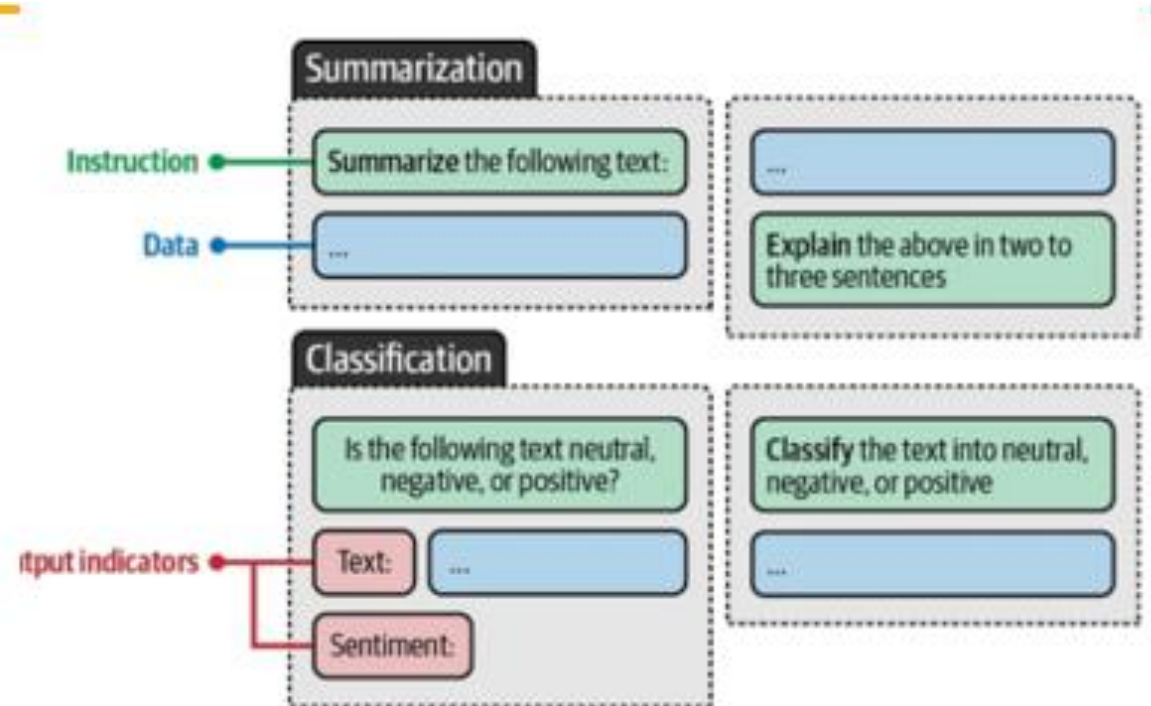


Figure 6-8. Extending the prompt with an output indicator that allows for a specific output.





Anatomy of a Well-Engineered Prompt

Role / Persona

Who the model should be — sets tone, expertise, vocabulary. “You are a senior tax consultant...”

Instruction / Task

What to do, precisely: classify, extract, rewrite, compare.
One primary task per prompt

Context

Background the model needs: domain facts, retrieved documents, conversation state, constraints

Examples (Shots)

Input → output
demonstrations that define the task by example (few-shot / in-context learning)

Output Format

Schema, length, language, JSON keys, allowed labels — makes output machine-consumable

Input Data

The actual item to process — clearly delimited (XML tags, triple quotes) so it is never confused with instructions

Zero-Shot vs Few-Shot Prompting



Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: I think the food was okay.

Sentiment: ...

One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**

Text: I think the food was okay.

Sentiment:

Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**.

Text: I think the food was great!

Sentiment: **Positive**.

Text: I think the food was horrible...

Sentiment: **Negative**.

Text: I think the food was okay.

Sentiment:

- GPT-3 result (Brown et al., 2020): a 175B model with k=32 examples in the prompt rivalled fine-tuned models on many benchmarks — no gradient updates, “learning” happens in the forward pass
- Few-shot helps most when the task format is unusual or labels are ambiguous; it helps least when the model is already instruction-tuned for that exact task

- Cost: those examples are reprocessed on every request
- Mitigation: prompt caching reuses the examples' KV-cache, so structure the prompt with stable content (examples) first, variable content (query) last

In-Context Learning — What Is Actually Happening?

- In-context learning means the model adapts to a new task using *only the examples in the prompt* — its weights never change. The model isn't "learning" in the training sense; the examples simply steer the frozen model into the right behaviour for this task, for this one request.
- ICL is surprisingly sensitive to *how* you present the examples. Three well-documented biases:
 - **Example order** — the same examples in a different order can change the answer.
 - **Recency bias** — the *last* example carries extra weight, nudging the model toward its label.
 - **Majority-label bias** — if most examples share one label, the model over-predicts that label.
- In practice: **select examples by semantic similarity to the current query** (retrieve the most relevant shots rather than using fixed ones), and **keep formatting identical** across all examples. These two choices dominate accuracy.

Chat template



- Roles examples
 - “user” for messages sent by the user
 - “assistant” for messages sent by the model
 - “system” sent at the beginning of the conversation and include directives about how the model should behave in the subsequent chat
- Role information is usually injected by adding control tokens between messages, to indicate both the message boundary and the relevant roles
- different models have been trained with wildly different formatting and control tokens for chat hence chat templates are defined
- Chat conversations as a list of dictionaries, contains role and content keys, and represents a single chat message

```
{ "role": "user", "content": "Hello, how are you?" }
{ "role": "assistant", "content": "I'm doing great. How can I help you today?" }
{ "role": "user", "content": "I'd like to show off how chat templating works!" }
```

apply_chat_template() - Local Model Inference



```
messages = [  
    {"role": "system", "content": "You are a helpful assistant"},  
    {"role": "user", "content": "Hello!"}  
]  
  
# Convert to model-specific format  
prompt = tokenizer.apply_chat_template(  
    messages,  
    tokenize=False,  
    add_generation_prompt=True  
)  
  
# Then tokenize and generate  
inputs = tokenizer(prompt, return_tensors="pt")  
outputs = model.generate(**inputs)
```

- Formats conversation messages into the proper string format expected by a specific model's training.
- Method on the tokenizer object
- Used with locally loaded models (via transformers)
- Converts structured chat format to model-specific prompt string
- Each model has its own chat template (Llama uses different format than Qwen or Mistral)
- Returns a formatted string or tokenized input
- Requires manual generation call afterward

chat_completion - API-Based Inference



```
from huggingface_hub import InferenceClient

client = InferenceClient(token="hf_****")

completion = client.chat_completion(
    model="meta-llama/Llama-3.1-8B-Instruct",
    messages=[
        {"role": "system", "content": "You are helpful"},
        {"role": "user", "content": "Hello!"}
    ],
    max_tokens=100,
    temperature=0.7
)

print(completion.choices[0].message)
```

- OpenAI-compatible API endpoint for hosted model inference
- Used with hosted models (Inference API/Endpoints)
- Follows OpenAI's chat completion format
- No local resources: Doesn't require GPU/RAM for model
- Faster setup: No model download/loading time
- Scalability: Can handle many concurrent requests
- Simpler code: No tokenization/generation logic needed

Delimiters



Delimiter Type	Syntax	Primary Use	Example
Triple Quotes	"""..."""	Text blocks, paragraphs	"""Summarize this text..."""
Triple Backticks	```...```	Code blocks	```python code here```
XML Tags	<tag>...</tag>	Semantic sections	<context>...</context>
Square Brackets	[LABEL]...[/LABEL]	Named sections	[CONTEXT]...[/CONTEXT]
Curly Braces	{...}	Variables, placeholders	{user_input}
Angle Brackets	<...>	Placeholders	<insert name here>
Triple Dashes	---	Section separators	---
Triple Hashtags	###	Headers, section titles	### Instructions
Double Hashtags	##	Sub-sections	## Step 1
Pipes	`	`	Column separators
Asterisks	*...*	Emphasis, bullets	* Item 1
Equal Signs	===	Strong separators	===
Greater Than	>>>	Nested content	>>> quoted text

Chain-of-Thought: Think Before Answering

innovate

achieve

lead

One-shot prompt

Prompting with a single example

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The answer is 27. ❌

Chain-of-thought prompt

Prompting with a reasoning example

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls.
 $5 + 6 = 11$
The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$.
The answer is 9. ✅

Example

Reasoning process (thought)

Instruction

Reasoning process (thought)

Few-shot CoT: demonstrations include the intermediate reasoning, not just the answer — the model imitates the reasoning pattern at test time.

Where CoT helps: math, logic, multi-step QA, planning. Where it doesn't: simple retrieval/classification — it adds cost and can even distract.

Zero-shot chain-of-thought



Zero-shot chain-of-thought

Prompting without example

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Instruction

Let's think step-by-step.

Prime reasoning

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$.

Reasoning process (thought)

The answer is 9. ✓

Chain-of-thought prompting without using examples. Instead, it uses the phrase “Let’s think step-by-step” to prime reasoning in its answer.

Alternatives

- “Take a deep breath and think step-by-step”
- “Let’s work through this problem step-by-step.”

Self-Consistency: Improving Outputs via Multiple Samplings



- Generate same prompt multiple times with sampling enabled, collect all outputs, and use majority voting on final answer.
- Process (3 steps)
 - Sample N outputs from the same prompt
 - Use `temperature > 0` and `do\_sample=True` to introduce variability
 - Extract final answers from each sample
 - Parse the answer from each output (e.g., regex to find numbers)
 - Majority voting: pick the answer that appears most often

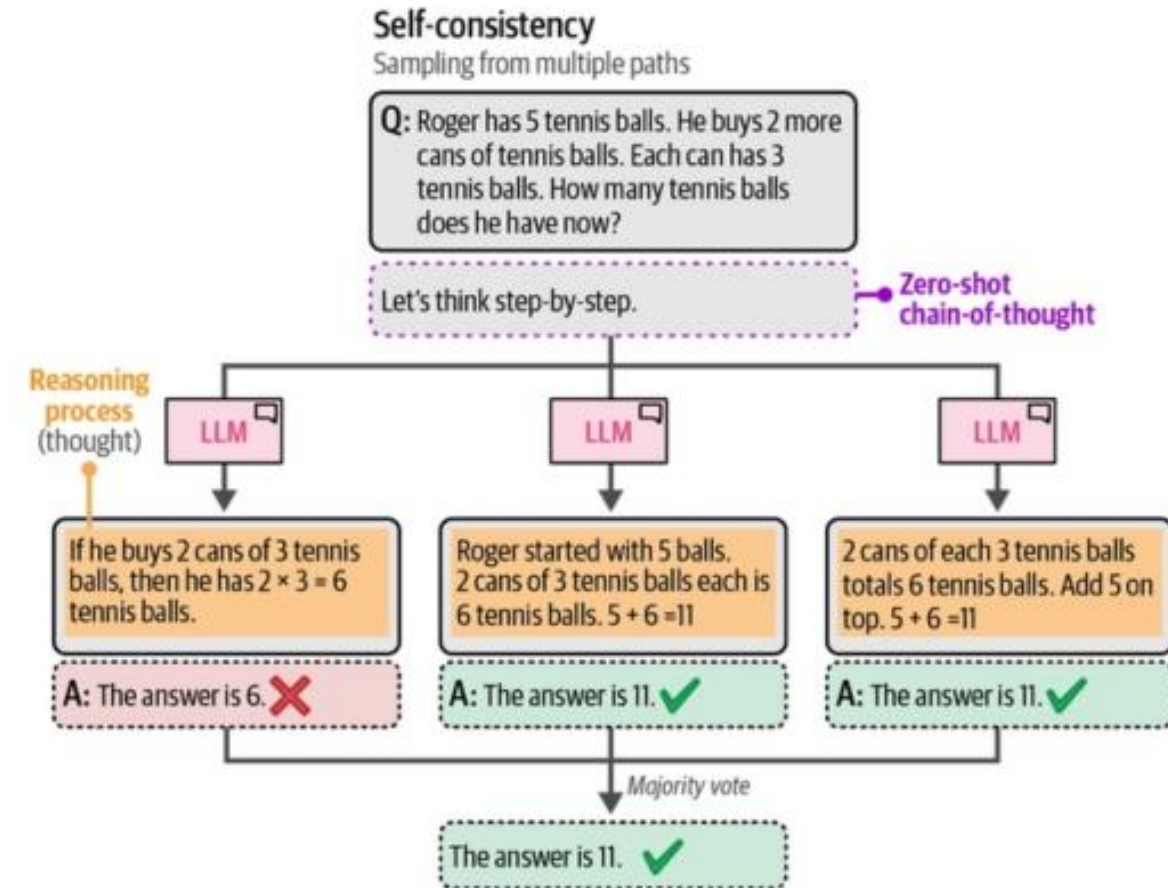
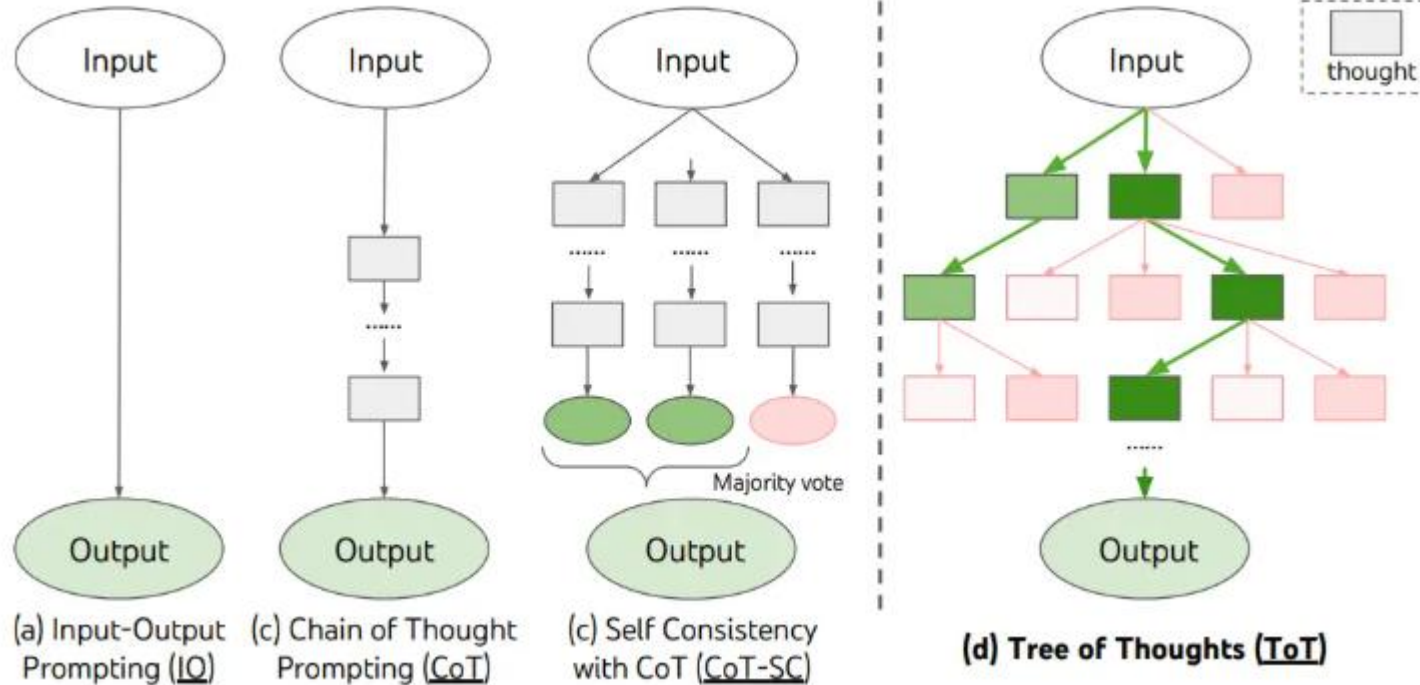


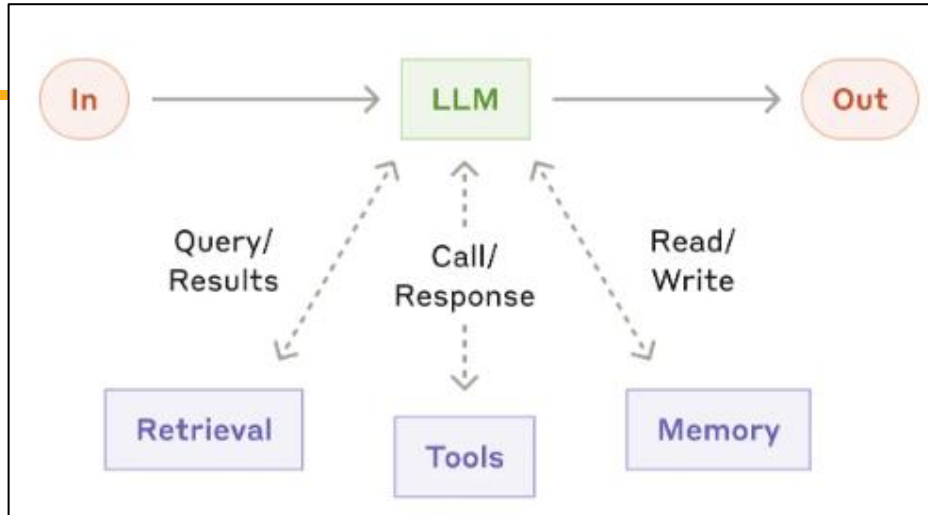
Figure 6-17. By sampling from multiple reasoning paths, we can use majority voting to extract the most likely answer.

Tree of Thought Prompting



- Why linear reasoning isn't always enough
- CoT produces ONE reasoning path:
Step 1 → Step 2 → Step 3 → Answer
- Tree of Thought
 - Exploring multiple reasoning paths (like a search tree)
 - At each intermediate step, generate multiple candidate thoughts (branches)
 - Evaluate each candidate (which looks most promising?)
 - Select the best candidates and continue expanding from them
 - Prune unpromising branches to save computation
 - Result: A tree of reasoning paths, not a linear chain

From Prompts to Pipelines



<-- Augmented LLM

Prompt as system component

- Each LLM call does ONE job with ONE schema
- Control flow (if/else, loops, retries) lives in code, not in the prompt
- Every stage is independently testable & swappable (model per stage)
- Observability: log inputs/outputs per stage

Prompt as artifact

- One mega-prompt tries to do everything
 - Failures are opaque — which part broke?
 - Hard to test, version, or improve locally
 - Accuracy degrades as instructions pile up
-
- Four design tools : chaining (sequential stages), decomposition (split the query), routing (pick the path), and function calling (let the model invoke code)
 - LangChain / LangGraph, LlamaIndex, DSPy — but the patterns are framework-independent

Prompt chaining

- Breaking Problems into Sequential Steps
- Traditional approach
 - One complex prompt → One output
- Each step in the chain is modular and independently debuggable
- Ideal for situations where the task can be easily and cleanly decomposed into fixed subtasks

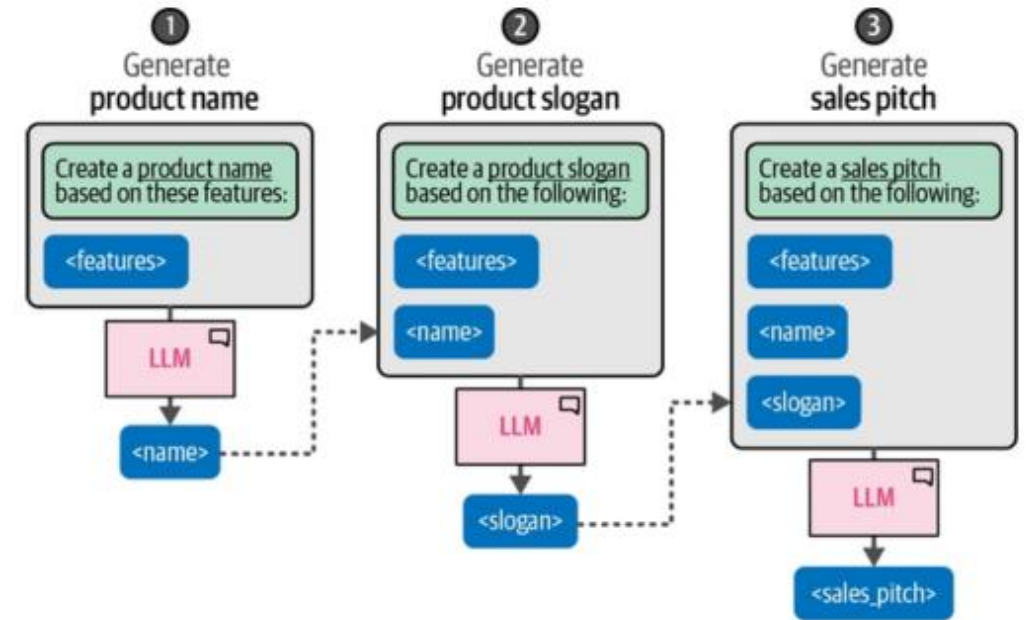
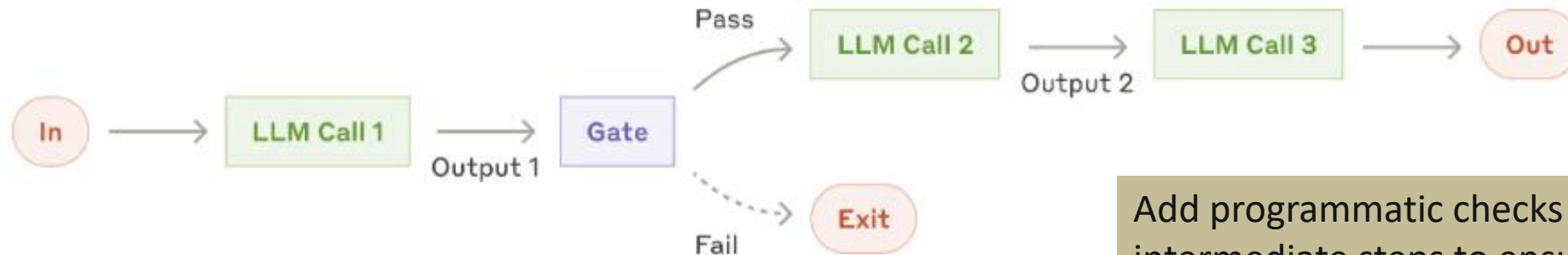


Figure 6-14. Using a description of a product's features, chain prompts to create a suitable name, slogan, and sales pitch.



Add programmatic checks ("gate") on any intermediate steps to ensure that the process is still on track.

Chain with Multiple Prompts

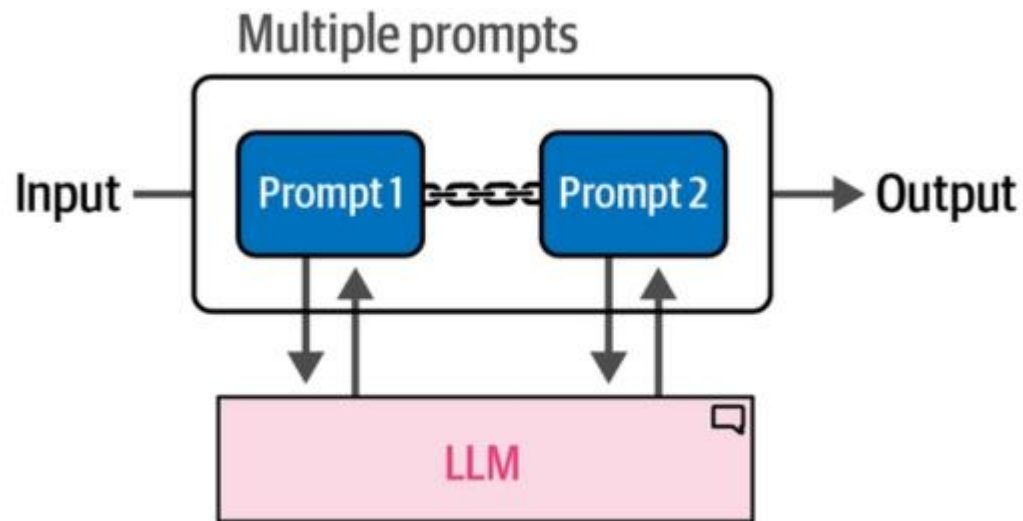


Figure 7-7. With sequential chains, the output of a prompt is used as the input for the next prompt.

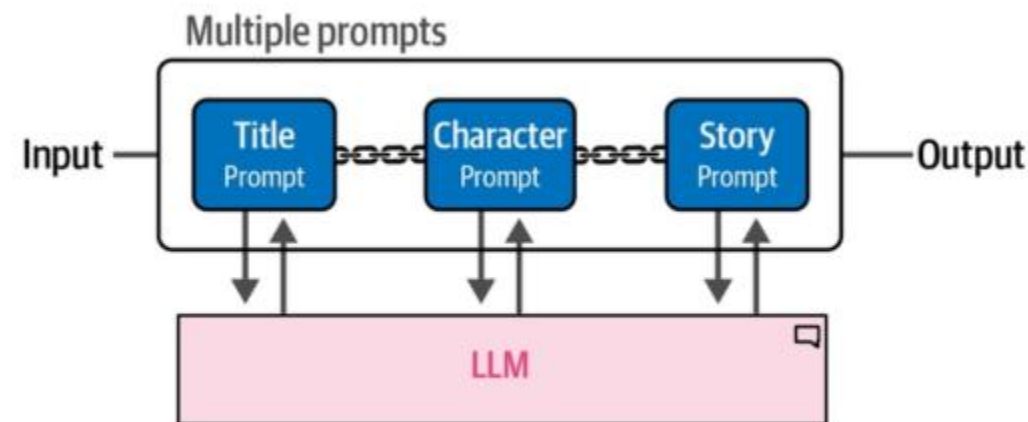
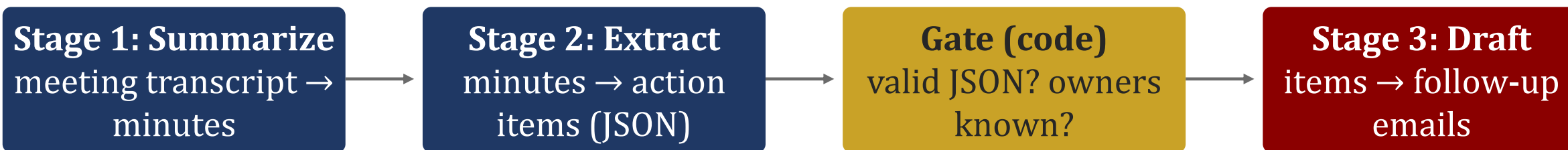


Figure 7-8. The output of the title prompt is used as



on failure: retry / re-ask (loop back)

LangChain chain



Prompt chaining : sequence LLM steps so each output feeds the next.

LangChain is one framework that implements it — a 'chain' is a reusable pipeline connecting a prompt, a model, and a parser

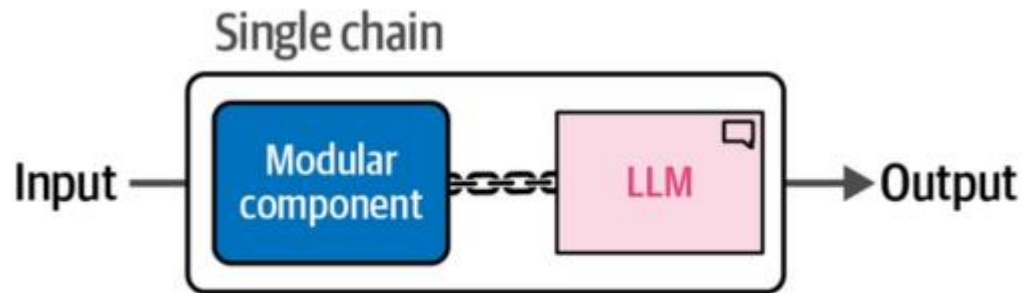


Figure 7-3. A single chain connects some modular component, like a prompt template or external memory, to the LLM.

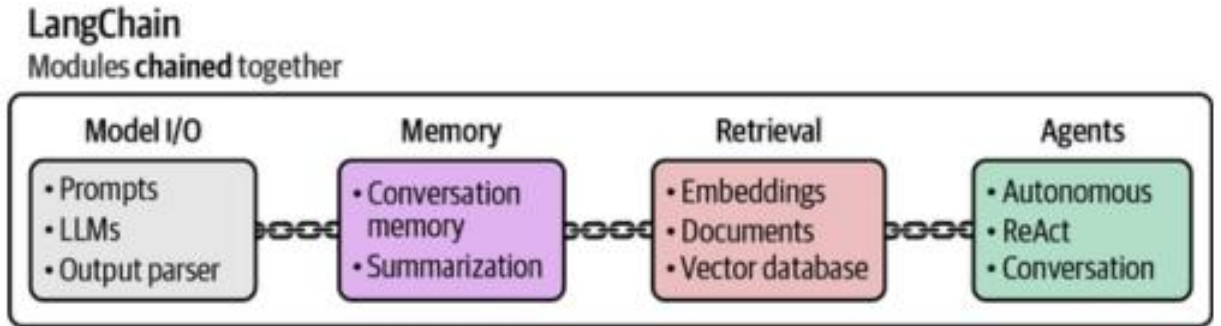


Figure 7-1. LangChain is a complete framework for using LLMs. It has modular components that can be chained together to allow for complex LLM systems.

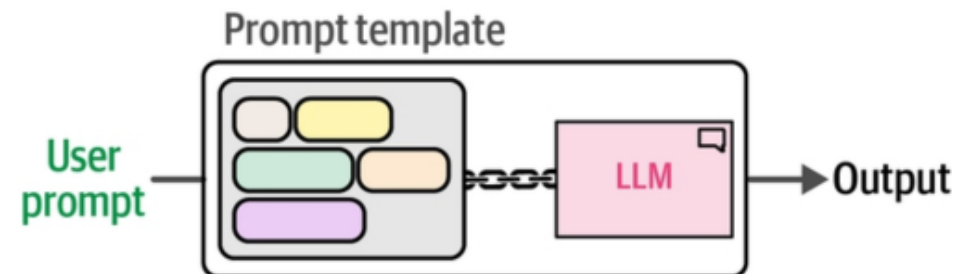


Figure 7-4. By chaining a prompt template with an LLM, we only need to define the input prompts. The template will be constructed for you.

combine achieve lead

- ## DECOMPOSER PROMPT → OUTPUT → PIPELINE

Break the question into 2-3 sub-questions that can each be answered independently. Output a numbered list only, nothing else.

CALL 1 OUTPUT → drives retrieval

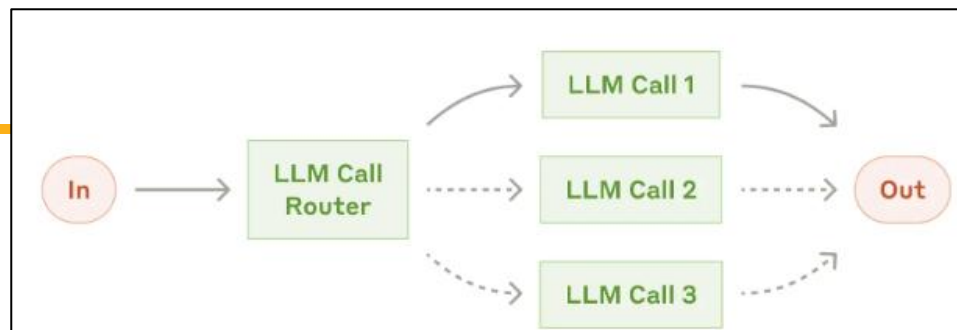
- ## # CALLS 2-4 – PIPELINE

```

Q1 → retrieve+answer  ┌
Q2 → retrieve+answer  ┤ → synthesize
Q3 ← uses Q1,Q2 answers ┘

```

Routing / Query Classification



LLM-AS-ROUTER PROMPT

You are a router for a telecom assistant.
Classify the query into exactly one route:

- BILLING payments, invoices, refunds
- TECH connectivity, device, outage
- SALES new plans, upgrades
- HUMAN legal threats, churn risk,
 anything you are unsure about

Return ONLY: {"route":"...",
 "confidence":0.0-1.0}

Query: "Internet down since morning and
your app shows a bill for a closed
account!"

→ {"route":"HUMAN","confidence":0.72}

- A router is a cheap classifier in front of expensive paths: send each query to the right prompt, tool, index, or model.
- Implementations: (a) LLM-as-router — a small/fast model with a classification prompt; (b) embedding/semantic router — nearest-route by cosine similarity (no LLM call); (c) fine-tuned classifier for high volume.
- Routing dimensions: intent (billing vs tech), difficulty (small model vs reasoning model — cost routing), safety (escalate to human), data source (which index).
- This workflow allows for separation of concerns, and building more specialized prompts.

Function-calling



- Extend LLMs with custom capabilities by enabling them to call external functions based on *natural language instructions + function descriptions* and output a string that invokes that function
- **LLMs only *generate* function call strings—they don't actually execute them**
- Function Calling: LLM capability to generate function call strings
- Tools: Actual functions being called by the LLM
- Function Description: Text explaining what the tool does
- Function Prototype: Function signature in the expected format (Python, JSON, etc.)
- General-Purpose LLMs: Respond to all query types (including function calling)
- Special-Purpose LLMs: Fine-tuned for specific tasks like function calling
 - Example: NexusRavenV2-13B, gorilla-openfunctions-v2 (Based on Llama 3), functionary-small-v3.1 (based on Llama 3)

Tools – Functionality We Give the LLM



- Standard LLMs are "Closed-Box" systems—they only know what they were trained on
- Traditional LLM: "I think the weather in Paris is usually mild in May." (Stale Data)
- Tool-Enabled LLM: "I don't know the current weather, but I see a get_weather tool. I will call it." (Real-time Data)
- Tools are not "installed" into the model; they are described to it
- Model decides *if and when* to use tools; your app executes them.
- Example Tools
 - get_weather — retrieve current weather for a location
 - get_account_details — look up user account by ID
 - issue_refund — process a refund for an order
 - search_docs — query a knowledge base or database

Tools – Functionality We Give the LLM

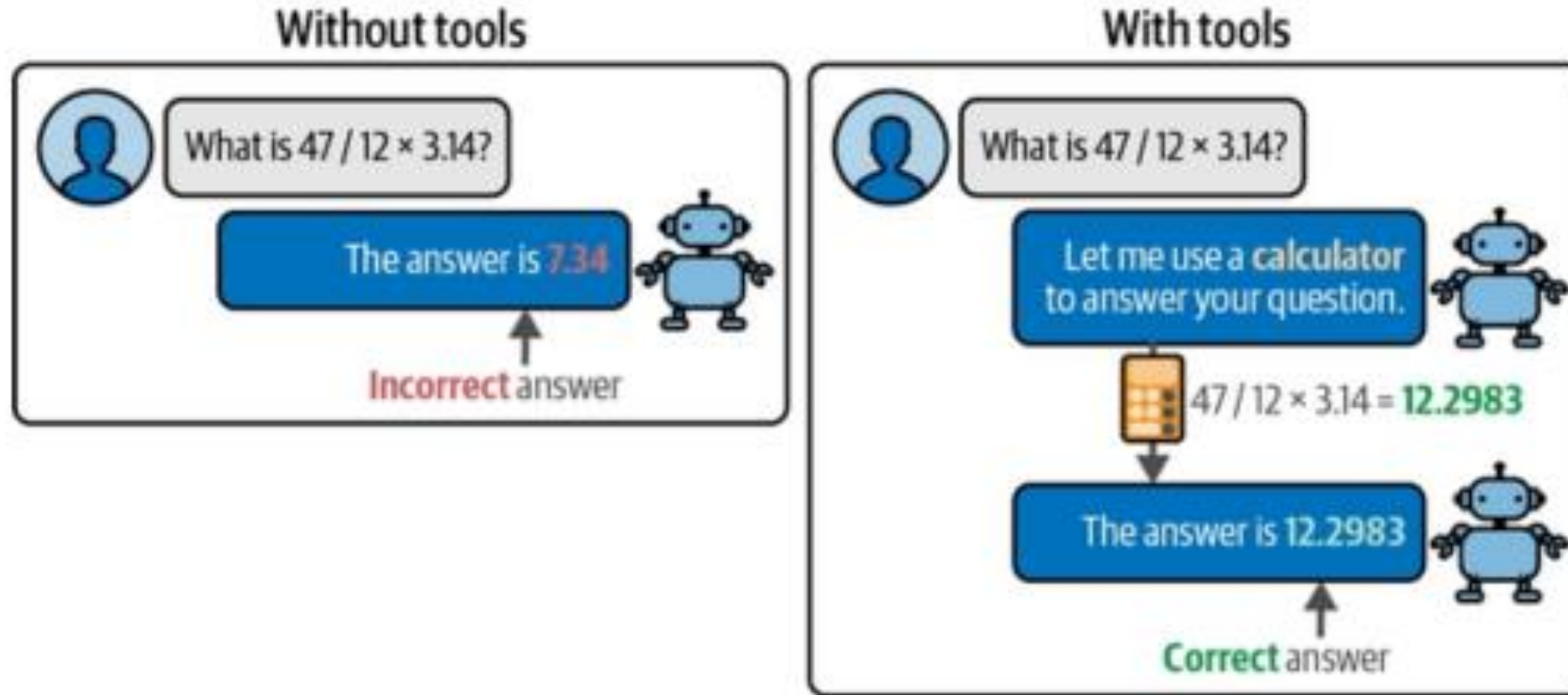
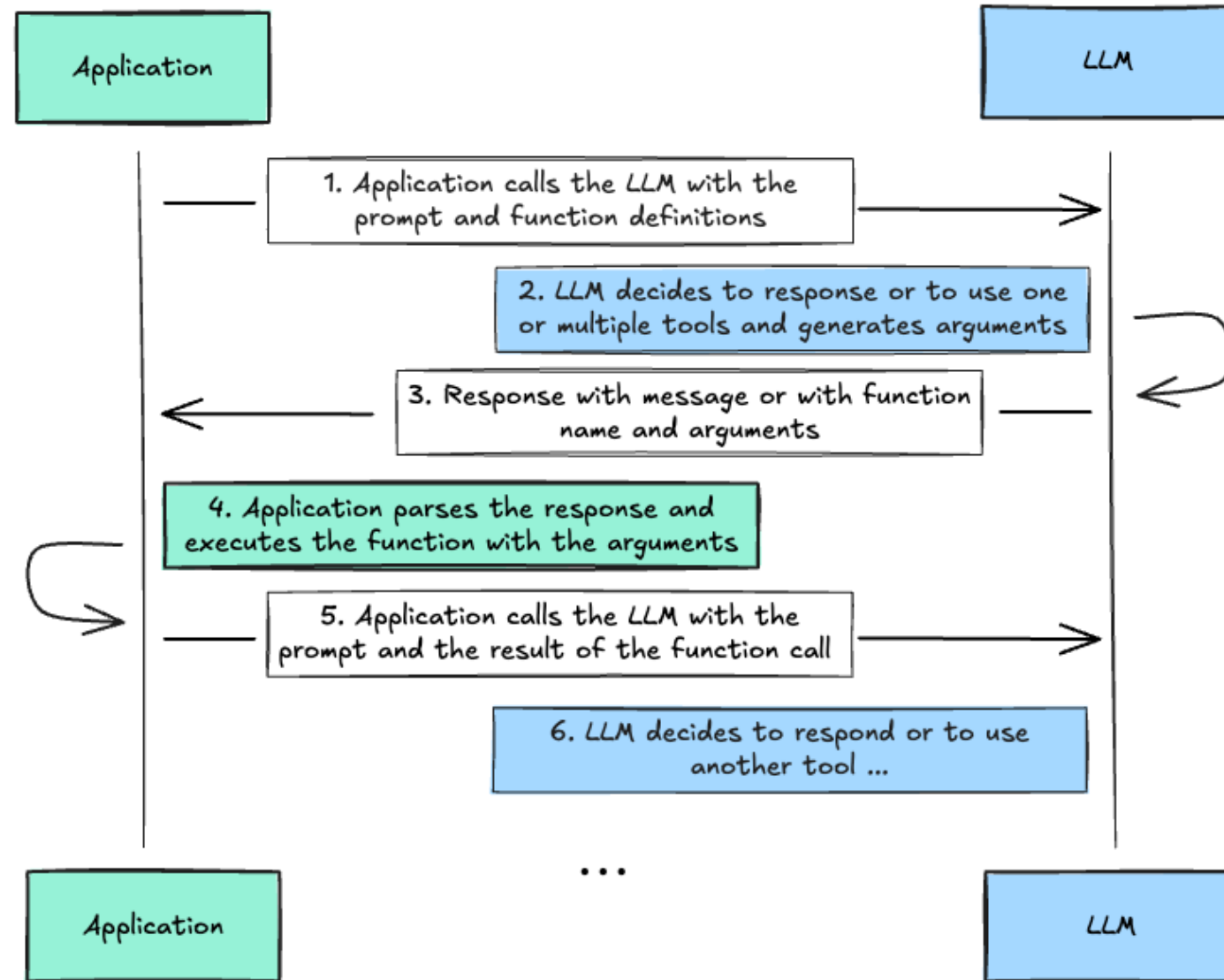


Figure 7-14. Giving LLMs the ability to choose which tools they use for a particular problem results in more complex and accurate behavior.

Function-calling



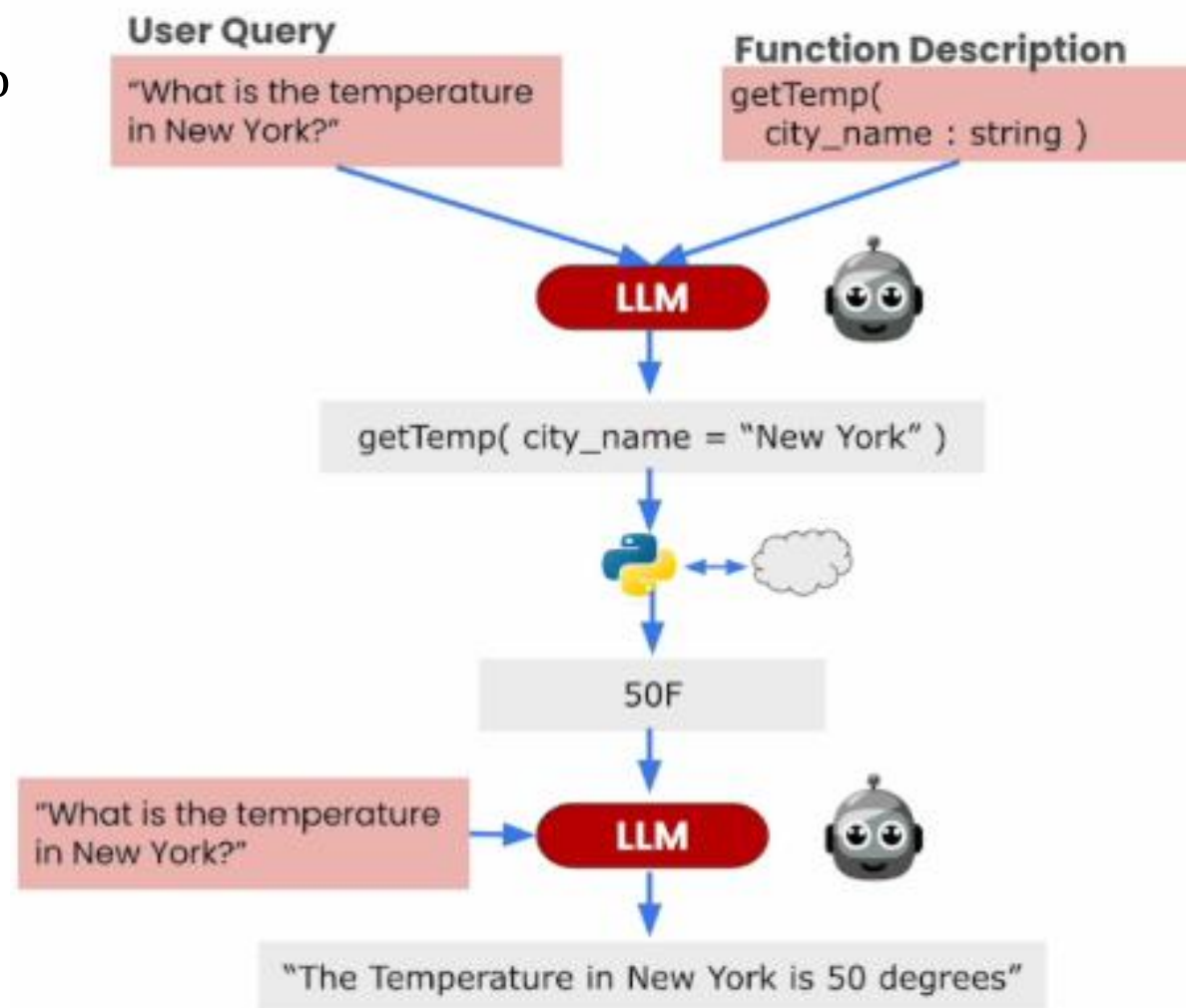
Using Tools (Function Definitions)

- Tools are the primary way to define callable functions for your LLM. Each tool requires:
- A unique name
- A clear description
- A JSON schema defining the expected parameters

Function Calling Workflow

- Step 1: Provide user query + function description to LLM
- Step 2: LLM recognizes it can use the function to answer
- Step 3: LLM generates executable function call string
- Step 4: System executes the function
- Step 5: Return results and original query to LLM
- Step 6: LLM generates final answer

What Is Function Calling?



Parallel function calling



- Specialized capability where an LLM generates multiple tool requests in a single response
- LLM outputs an array of tool calls in a structured format (usually JSON), e.g

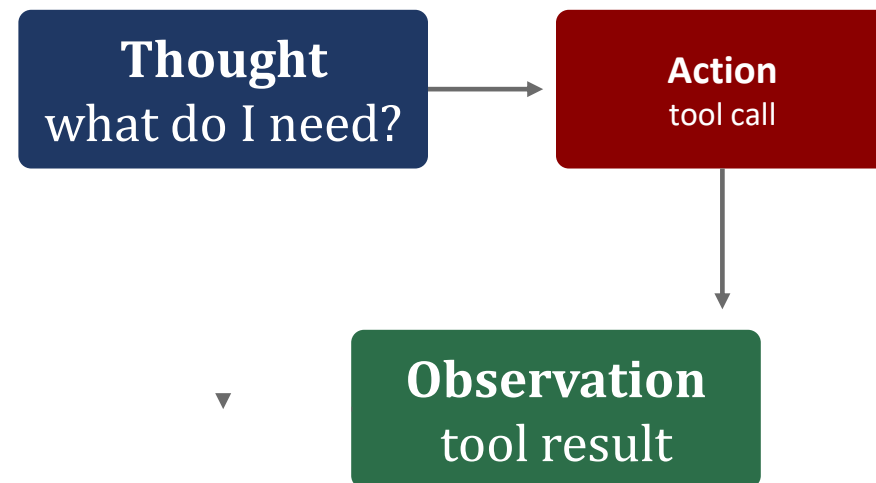
```
{
  "tool_calls": [
    {
      "id": "call_001",
      "type": "function",
      "function": { "name": "get_weather", "arguments": "{\\"location\\": \\"London\\"}" }
    },
    {
      "id": "call_002",
      "type": "function",
      "function": { "name": "get_weather", "arguments": "{\\"location\\": \\"Tokyo\\"}" }
    }
  ]
}
```

Iterative reasoning



ReAct — Reasoning + Acting (Yao et al., 2023)

- Interleave three message types in a loop:
Thought (free-form reasoning about what to do next), Action (a tool call: search, lookup, calculator, API), Observation (the tool's result, appended to context).
- Reasoning guides acting (decide what to look up); acting grounds reasoning (real evidence replaces hallucinated facts).
- **ReAct is the founding pattern of today's tool-using agents**



loop until Thought concludes "I now know the answer" → Final Answer

ReAct

innovate

achieve

lead

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod



(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch



(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control ...

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ...

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

Act 4: Finish[yes]



(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: Finish[keyboard function keys]



Iterative reasoning



- **Step-Back Prompting**
 - Before answering, the model first derives the general principle or higher-level concept behind the question, then uses it to answer the specific case
- **Self-Reflection / Critique**
 - model reviews its own first output, critiques it, and produces a revised version, Expensive

Structured Data Handling



- Input
 - Tables → LLM (input): how do you get structured data into the model?
 - Serialize, image, or query
- Output
 - LLM → JSON (output): how do you get structured data out of the model reliably?
Manual, JSON mode, constrained decoding

Handling Tables with LLMs



- A table isn't text. Before an LLM can use one, you choose *how it enters the model*, **3 paths**
- *Serialize into the prompt*
 - Extract → render as text (Markdown / CSV / JSON / key-value) → put in prompt
 - Best for: small tables the model must reason over as a whole
 - Cost: the format choice affects accuracy *and* tokens
- *Feed as an image*
 - Skip extraction; hand the page/table image to a multimodal model
 - Best for: messy, merged-cell, visually-complex tables
 - Cost: higher token cost; no clean structured copy of the data
- *Query the table*
 - Extract to DataFrame / SQL → LLM writes code or SQL → sees only the answer
 - Best for: large tables; specific facts or aggregates
 - Cost: needs a tool/execution step,

Tabular Serialization — Feeding Tables to an LLM



- LLMs read token streams — a table must be serialized to text. The choice of serialization measurably changes accuracy on table-QA tasks.
- Markdown / pipe tables: compact, well represented in pretraining; good default for small-to-medium tables.
- JSON records (row-wise dicts): verbose but robust — column-value binding is explicit, best when rows are processed individually.
- CSV: cheapest in tokens but column alignment is implicit — errors grow with wide tables.
- Practical rules: keep header row, prune unused columns, chunk long tables, and state units in headers (“revenue_inr_cr”).

SAME TABLE, THREE SERIALIZATIONS

Markdown (good default)

qtr	revenue_inr_cr	margin_pct
Q1	142.5	18.2
Q2	156.1	19.0

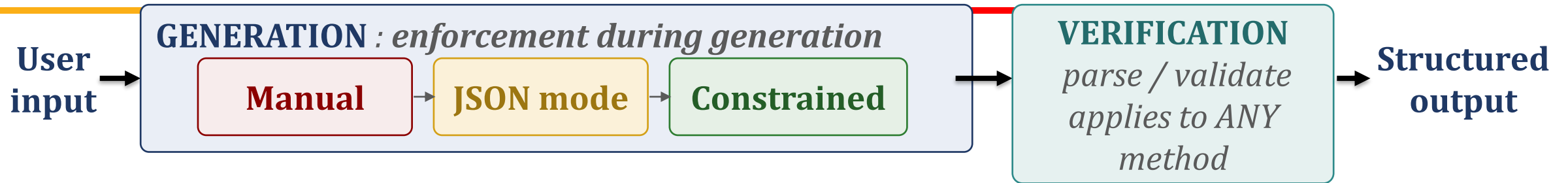
JSON records (explicit binding)

```
[{"qtr": "Q1", "revenue_inr_cr": 142.5, "margin_pct": 18.2}, {"qtr": "Q2", "revenue_inr_cr": 156.1, "margin_pct": 19.0}]
```

CSV (cheap, fragile when wide)

```
qtr,revenue_inr_cr,margin_pct
Q1,142.5,18.2
```

Structured Output : Generate and Verify



Method	Also called	Enforcement · grammar	Guarantee
Manual (prompt + parse)	Output Verification	After generation · no grammar	None
Fine-tuning	(training-based)	In the weights · training time	Soft — one format, costly to change
JSON mode	Structured Generation	During · generic JSON grammar	Valid syntax, not schema
Constrained decoding	Constrained Decoding	During · schema-specific FSM	Hard schema compliance

Constrain generation to guarantee the **shape**; verify output to guarantee **meaning**. Robust systems do both.

Output Verification



What to Check

L1

Syntactic Does it parse at all? (valid JSON / expected format)

How: a parser, e.g. `json.loads` · deterministic

L2

Schema / structural Right keys, types, enums, required fields?

How: JSON Schema / Pydantic · deterministic

L3

Semantic / business rules Are the values actually correct & safe? (dates in range, totals add up, IDs exist, content accurate)

How: coded rules where possible, else a model

How

Deterministic (code)

fast · free · reliable

- Schema validation (Pydantic)
- Regex / pattern matching
- Type & range checks
- Whitelist / blacklist

Model-based (LLM)

for judgments code can't express

- Self-verification
- Chain-of-Verification (CoVe)
- Verifier agent (separate model)
- LLM-as-checker

Rule of thumb: push each check to the cheapest layer that can catch it — **code before model**.
Constrained decoding already guarantees L1–L2, so verification then focuses on **L3 (semantics)**.

Manual (prompt + parse)



"After generation · no grammar · Guarantee: None." And the lab's result is the honest advertisement of that row's "None" guarantee — even the best-structured prompt (`delimited`) still produced off-enum sentiment labels like `"Neutral/Positive"`, because prompting is a request, not a constraint.

Component	Lab 1.2
Manual generation	The three prompt builders (prose, numbered, delimited) — all just ask for JSON in different structural styles. Nothing constrains the decoder
Verification (parse)	<code>try_parse_json()</code> — brace-match and <code>json.loads()</code>
Verification (validate)	<code>is_reliable()</code> / <code>score_response()</code> — check keys present, enums valid, <code>order_number</code> digits-only

Lab 1.2's `try_parse_json` is L1. `is_reliable` / `score_response` are L2 (keys present, sentiment in enum, urgency in enum, `order_number` digits-only). There's no L3 check in the lab — we never verify that the extracted `product` matches what the message actually names, or that the `issue` is faithful.

What to do when a check fails : Validate-and-Repair Loop



- When a check fails, don't discard the output — **feed the error back**. The validation message is itself a prompt, and a *specific* error repairs far better than “invalid, try again”.
- **pattern**
 - Generate the output from the LLM.
 - Parse & validate against the schema (Pydantic).
 - On failure, re-prompt with the exact error appended.
 - Cap retries (e.g. 3 attempts).
 - Still failing? Route to a fallback.
- Works with any model — no special API
- Latency only on failures (success = one call)
- Layers on top of JSON mode or constrained decoding
- **Libraries:** instructor · guardrails-ai · LangChain parsers.

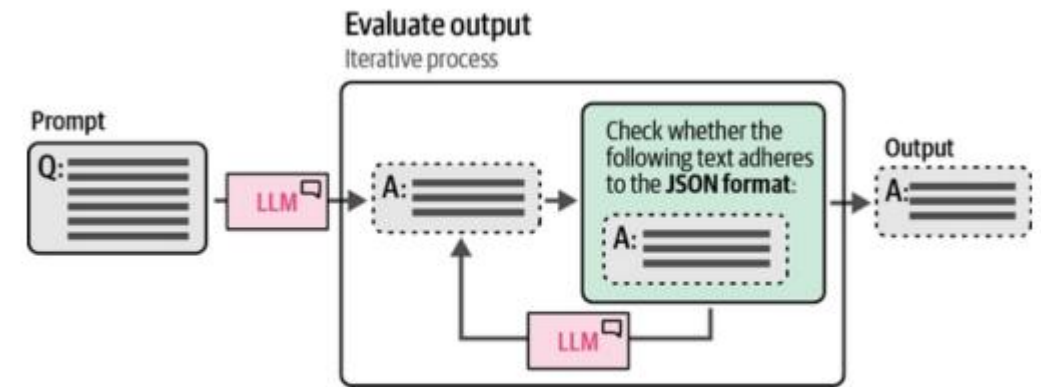


Figure 6-19. Use an LLM to check whether the output correctly follows our rules.

Constrained decoding



- Mechanism: compile the schema/grammar (regex, CFG, JSON Schema) into a finite-state machine over tokens. At each step, tokens that would leave the valid language are masked to $-\infty$ before sampling.
- Three main ways that a structured output can be defined for constrained decoding: regex, code (Pydantic models), and generative constructs.
- Some recent models are already fine-tuned to incorporate these constraints inherently, reducing the need for extensive post-processing.
- Example applications: automated content creation, data validation, and real-time language translation
- When models are not inherently fine-tuned for constrained decoding, or when more precise control over the output is desired, dedicated libraries like *LM Format Enforcer* and *Outlines* provide tools to enforce specific constraints on model outputs

Grammar-Constrained Sampling



- Restrict token generation to only grammar-valid sequences
- Guarantees syntactically valid output (e.g., valid JSON, SQL)
- Filter next-token distribution based on grammar at each decoding step

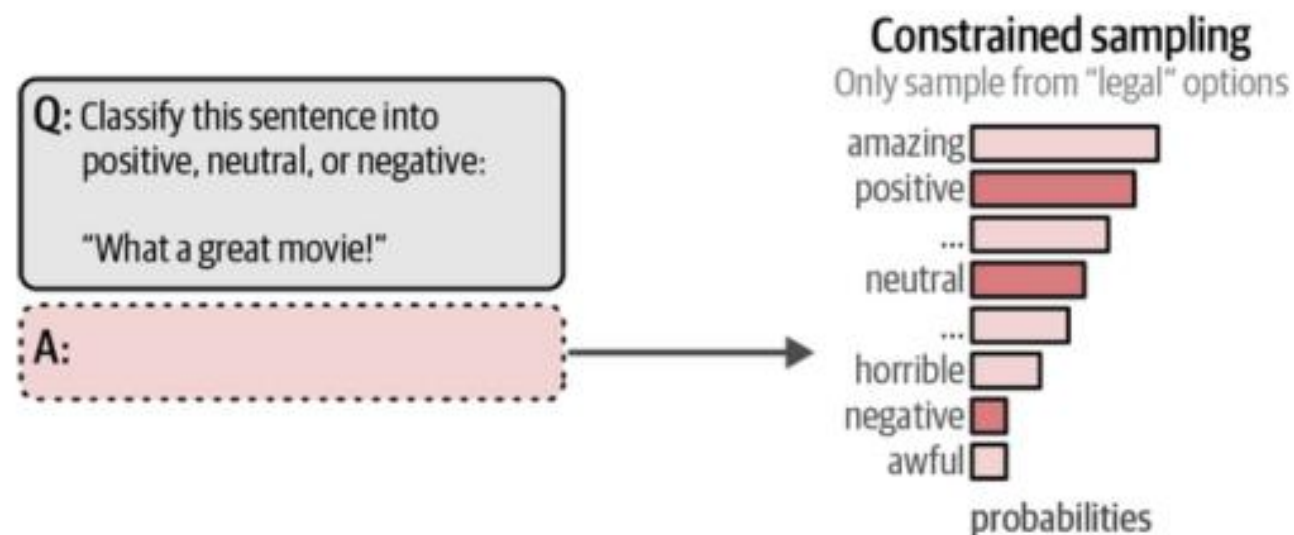
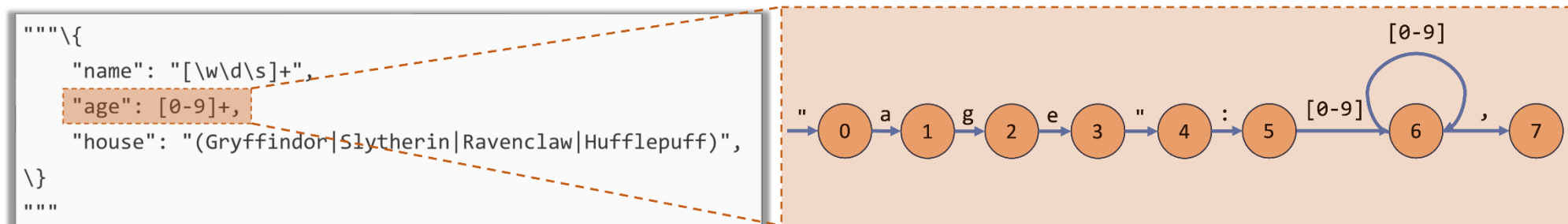


Figure 6-21. Constrain the token selection to only three possible tokens: "positive," "neutral," and "negative."

Constrained decoding



schema $\rightarrow$ regex $\rightarrow$ finite-state machine $\rightarrow$ token masking



Regular Expression

Finite State Machine

Please fill in the following information about Harry Potter.

```
{
  "name": "Harry",
  "
```

Decoding Status

Decode + FSM

- age ✓
- Age ✗
- hou ✗

Please fill in the following information about Harry Potter.

```
{
  "name": "Harry",
  "age":
```

Decoding Status

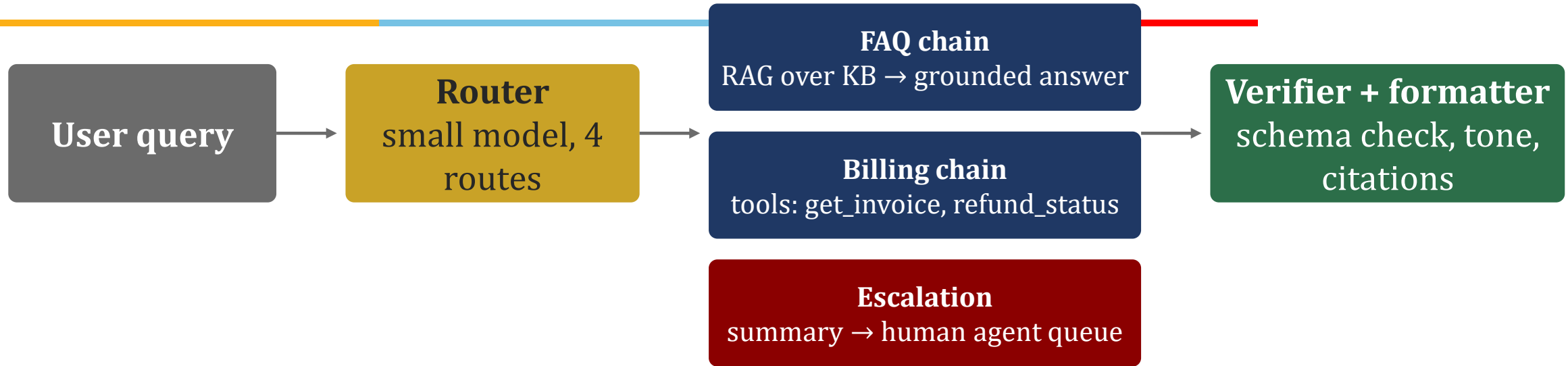
Decode + FSM

- 0 ✓
- 1 ✓
- fif ✗

✓ allowed next token
✗ not allowed next token

Constrained Decoding With Logits Mask

Case Study — Customer-Support Copilot, End to End



Pattern	Blocks
Routing / Query Classification	Router block (4-way dispatch)
Function-Calling	Billing chain (get_invoice, refund_status)
Query Decomposition	Inside the billing chain (bill-higher → fetch → diff → explain)
Prompt Chaining & Control Flow	FAQ chain, Escalation branch, and the overall query→...→verifier flow
Structured Output Control	Verifier + formatter — cross-link to the other section

Automated Prompt Optimization



- Instead of hand-tuning prompts — **optimize them programmatically** against a metric or labelled eval set. Prompts become parameters, tuned by data rather than intuition
- **APE — Automatic Prompt Engineer**
 - An LLM generates many candidate instructions, scores each on your eval set, and keeps the best one. *Reference repo exists — in practice used via DSPy's MIPRO optimizer.*
- **OPRO — Optimization by PROMpting**
 - The LLM acts as an optimizer: it proposes better prompts iteratively, using the scores of past attempts as feedback. *Research code only — lives on as DSPy's COPRO optimizer.*
- **DSPy / GEPA — the framework that operationalizes the above**
 - Declare the pipeline; it compiles and optimizes the prompts and few-shot examples automatically against your metric. *MIPROv2 = current default optimizer; GEPA = the newer reflective one.*

Test-Time Compute — Thinking Longer



- Two ways to make a model better: the old way — a bigger model trained on more data — and a new way: let the model spend more compute thinking about each question, at answer time.
- You set a thinking budget: more thinking gives better answers, but costs more time and money. You choose how much, per task.

Parallel — best of many

Ask the model several times, then pick the best answer. Easy to add — but you need a way to check which answer is right.

Sequential — one long think

The model writes one long reasoning trace — it plans, tries, catches its own mistakes, then answers. These are its “thinking tokens.”

Reasoning models (OpenAI o-series, Claude extended thinking, Gemini thinking, DeepSeek-R1) do this automatically — trained in, not prompted.

Prompting change: with these models, stop adding “think step by step.” Just state the problem, the constraints, and what a good answer looks like — and let the model reason on its own.

From Prompt Engineering to Context Engineering

Prompt engineering optimizes **what you say**. Context engineering optimizes **everything the model can see** when you say it.

CONTEXT ENGINEERING

everything in the context window

PROMPT

the instruction you write

- + Retrieved documents
- + Memory & conversation history
- + Tool / function results
- + Schema & output format
- + Few-shot examples

Where to spend your effort

Bad output? First ask **“Did the model have the information it needed?”** (context) — before **“Did I phrase it well?”** (prompt)

→ Next module: RAG

Retrieval — choosing the right documents and placing them in the window — is the **largest piece of context engineering**. That is what RAG is.

Sample Question : Case study - **Scenario**



"HelpHub" is the customer assistant for an online electronics retailer. Customer messages fall into four intents:

Order tracking — "where is my order?"

Returns / refunds — "I want to return this item"

Product questions — "what's the price / stock / specs of the X200?"

Complaints / legal threats — account-closure threats, legal language, high churn risk

The team has three tools, each a thin, typed boundary that the **application** executes (the model never touches the data store):

`get_order_status(order_id)` — looks up live order state

`start_return(order_id, reason)` — opens a return in the returns system

`lookup_product(sku)` — **queries the product database** and returns structured fields (price, stock, key specs)

Refund and return outcomes must be returned to internal systems as JSON.

The current build is **one large prompt** that tries to do everything. It is slow (always calls the largest model), sometimes answers product questions from the model's stale training data instead of the live database, and has no clean way to escalate angry / legal messages to a human agent.

Sample Question : Case study - **Scenario**



(a) Decompose HelpHub into a pipeline. Describe the stages and name the system-design pattern at each stage (routing, prompt chaining, query decomposition, function-calling).

Ans: Stop using one mega-prompt and instead make **each LLM call do exactly one job**, with control flow (if/else, loops, retries) living in code around the calls.

1. A cheap, fast model — the **router** — classifies the incoming message into one of four routes, and each route is a specialised path.

2. Order tracking → function-calling. The model is given the `get_order_status(order_id)` description, generates the call string with the order ID, the app executes it, and the model phrases the live result.

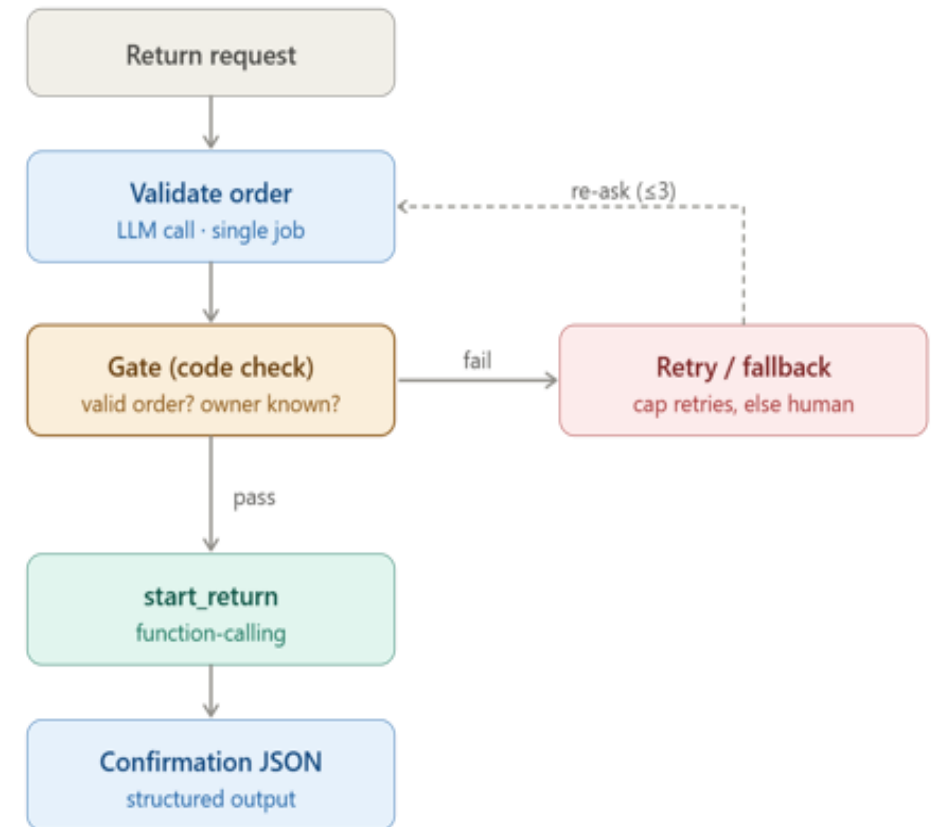
3. Product questions → function-calling (optionally with **query decomposition**). The model generates a `lookup_product(sku)` call; for a multi-part question ("compare price and battery of A vs B"), a decomposition step first splits it into focused sub-lookups.

Sample Question : Case study - **Scenario**



4. Returns / refunds → prompt chaining with a gate. A short sequence of single-job calls: validate the order → call start_return → produce the confirmation JSON. A **code gate** between stages checks the process is still on track (valid order? owner known?) before continuing; on failure it re-asks, with a capped number of retries, then falls back.

5. Complaints / legal → escalation branch. No tool, no clever prompt — routed straight to a human queue with a short LLM-generated summary. This branch also catches anything outside the four intents (the router's safe default)



Control flow (retries, branching) lives in code, not in the prompt.

Sample Question : Case study - **Scenario**



(b) For each of the four intents, state which tool / path it uses, and explain why a router improves **cost** and **reliability** versus the single mega-prompt.

Ans:

Cost: the router is a small, cheap classifier that sends each query down the lightest path that can handle it — easy intents never invoke the large model, which the mega-prompt always did.

Reliability: separation of concerns. Each route is a short, single-job prompt that is independently testable, loggable, and swappable, so a failure points at one specific stage. In the mega-prompt, intent detection, tool use, and formatting were tangled in one call, so failures were opaque — you couldn't tell which part broke.

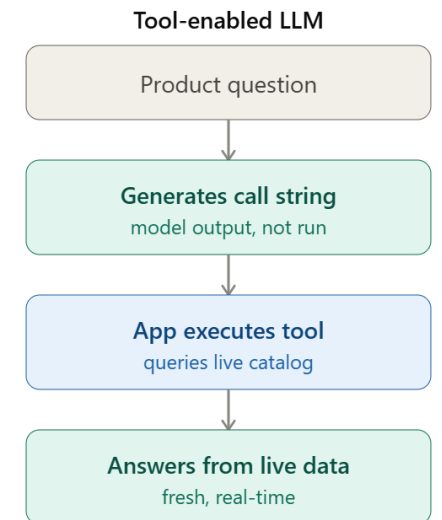
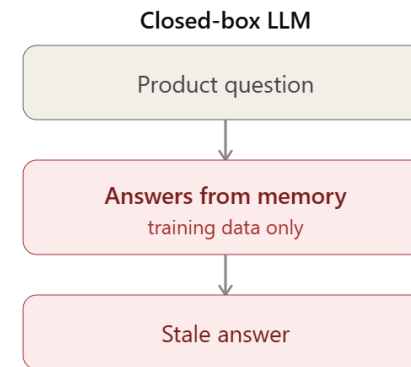
Intent	Path
Order tracking	function-calling — get_order_status
Returns / refunds	prompt chain → start_return + code gate
Product questions	function-calling — lookup_product(sku)
Complaints / legal	escalation to human queue

Sample Question : Case study - **Scenario**



(c) The product-question path must use the live database, not the model's memory. Using the closed-box vs tool-enabled distinction, explain what the model actually **generates** versus what **executes**, and how function-calling fixes the stale-data problem.

Ans: A standard LLM is **closed-box**: it only knows its training data, so it would answer product questions from stale memory. Function-calling makes it **tool-enabled**, and the key is the division of labour.



Sample Question : Case study - **Scenario**



- The model **only generates a function-call string** — the tool name plus JSON arguments, e.g. `lookup_product("SKU-X200")`. It does **not** execute anything.
- The **application** receives that string and runs the real database query (e.g. `SELECT price, stock, specs FROM products WHERE sku = ?`), then returns the structured row to the model, which composes the answer from that fresh result.
- Tools aren't "installed" into the model — they're *described* to it; the model decides when to call, the app does the calling.
- This fixes the stale-data problem because the answer is grounded in real-time data the app fetched, not whatever was frozen in the model's weights.
- Optional : a keyed lookup needs the exact SKU, but the customer typed "the X200," not "SKU-X200." So add a resolve step first — the model normalises the SKU, or a small `find_sku(name)` call precedes `lookup_product`.

Sample Question : Case study - **Scenario**



(d) Give one concrete failure the router itself can introduce, and one safeguard against it.

Ans:

Failure: misrouting. Because the router is a cheap classifier, it can send a query to the wrong path — most dangerously, classifying a legal threat or churn-risk complaint as a routine question, so it never reaches a human.

Safeguard: have the router return a confidence score and **default to the HUMAN route when confidence is low** ("anything you are unsure about → HUMAN"). A semantic / embedding fallback router, or an audit of low-confidence routes, also helps catch systematic misrouting.

Summary



Design

Six components; delimit inputs; schema-first output; serialize tables deliberately

System design

Chain small reliable calls; decompose queries; route by intent/difficulty; tools via function calling

Structured output

Verify-retry → schema-enforced APIs → constrained decoding (validity by construction)

Iteration

ReAct grounds reasoning in tool observations; reflection works only with external feedback

Paradigms

Zero/few-shot & ICL: demonstrations teach format & label space; self-consistency = sample & vote

Optimization

Prompts are hyperparameters: APE proposes & scores; OPRO optimizes in language; DSPy/GEPA compile pipelines

Reasoning

CoT gives scratch space; ToT searches; step-back abstracts; PAL delegates computation to code

Test-time compute

Accuracy is now a budget knob: best-of-N, long CoT, thinking budgets, difficulty routing

References



- <https://platform.openai.com/docs/guides/function-calling>
- <https://www.promptingguide.ai/techniques>
- <https://8738733.fs1.hubspotusercontent-na1.net/hubfs/8738733/eBooks/Weaviate-Context-Engineering-ebook.pdf>
- <https://ai.google.dev/gemini-api/docs/function-calling?example=meeting>
- Anthropic Prompt Engineering Guide
- OpenAI Prompt Engineering Guide
- Research papers
 - A Survey of Context Engineering for Large Language Models
 - Tree of Thoughts: Deliberate Problem Solving with Large Language Models
 - LARGE LANGUAGE MODELS ARE HUMAN-LEVEL PROMPT ENGINEERS
 - Unsupervised Question Decomposition for Question Answering
 - LEAST-TO-MOST PROMPTING ENABLES COMPLEX REASONING IN LARGE LANGUAGE MODELS
 - Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study
 - A Prompt Pattern Catalog
 - ICL as Implicit Bayesian Inference
 - Calibrate Before Use
 - REACT : SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS
 - Teaching LLMs to Self-Debug